

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

ОТЧЕТ

по Лабораторной работе № 6.2
«Работа с БД в СУБД MongoDB»

по дисциплине «Проектирование и реализация баз данных»

Обучающийся	Забродин Максим Александрович
Группа	К3240
Направление подготовки	09.03.03 Прикладная информатика
Преподаватели	Говорова Марина Михайловна, Белов Александр Олегович

Санкт-Петербург
2026

Цель работы

Овладеть практическими навыками выполнения CRUD-операций, работы с вложенными объектами, агрегирования и изменения данных, создания ссылок между документами и индексов в базе данных MongoDB.

Используемое программное обеспечение

Работа выполнена в операционной системе Windows 11 с использованием локального сервера MongoDB 8.3.4 и консольной оболочки MongoDB Shell (mongosh) 2.9.1. Сервер MongoDB установлен как служба Windows и доступен по адресу `mongodb://localhost:27017`.

Практическое задание

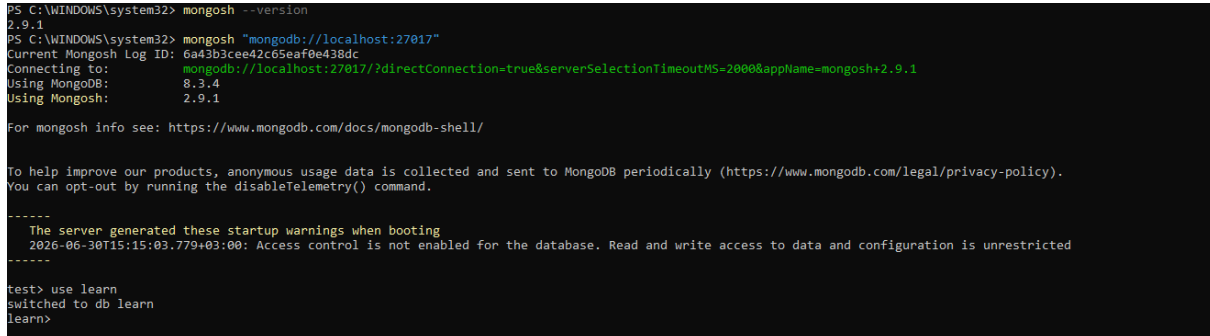
- создать базу данных `learn` и коллекцию `unicorns`;
- заполнить коллекцию документами и выполнить выборки с сортировкой, ограничением и проекцией полей;
- использовать логические операторы, работу с массивами и проверку наличия полей;
- создать коллекцию `towns` с вложенными объектами и выполнить запросы к вложенным полям;
- выполнить агрегированные запросы, обновление и удаление данных;
- организовать ссылки между документами и проверить их разыменование;
- создать уникальный индекс и проверить ограничение уникальности.

Выполнение

1. Подключение к MongoDB и создание базы данных

В PowerShell была проверена версия оболочки mongosh, после чего выполнено подключение к локальному серверу MongoDB. Командой use learn выбрана база данных learn. Предупреждение об отключённом контроле доступа допустимо для локальной учебной установки.

```
mongosh --version
mongosh "mongodb://localhost:27017"
use learn
```



```
PS C:\WINDOWS\system32> mongosh --version
2.9.1
PS C:\WINDOWS\system32> mongosh "mongodb://localhost:27017"
Current Mongosh Log ID: 6a49b3cee42c65eaf0e438dc
Connecting to:  mongodb://localhost:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.9.1
Using MongoDB:  8.3.4
Using Mongosh:  2.9.1

For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/

To help improve our products, anonymous usage data is collected and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

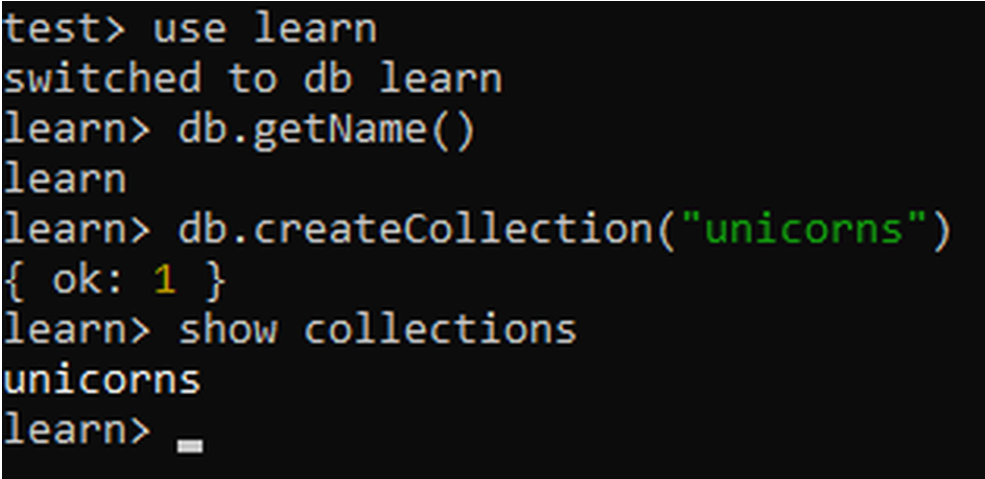
-----
The server generated these startup warnings when booting
  2026-06-30T15:15:03.779+03:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----

test> use learn
switched to db learn
learn>
```

Рисунок 1 – Подключение к локальному серверу MongoDB и выбор базы данных learn

После выбора базы была проверена её текущая идентификация и создана коллекция unicorns.

```
db.getName()
db.createCollection("unicorns")
show collections
```



```
test> use learn
switched to db learn
learn> db.getName()
learn
learn> db.createCollection("unicorns")
{ ok: 1 }
learn> show collections
unicorns
learn> _
```

Рисунок 2 – Создание базы данных learn и коллекции unicorns

2. CRUD-операции: вставка и выборка данных

2.1. Заполнение коллекции unicorns

Первым способом в коллекцию методом insertMany() были добавлены 11 документов, описывающих единорогов. Каждый документ содержит имя, массив предпочтений, вес, пол и количество убитых вампиров. У Nimue поле vampires намеренно отсутствует.

```
db.unicorns.insertMany([
  {name: "Horny", loves: ["carrot", "papaya"], weight: 600, gender: "m", vampires: 63},
  ...
  {name: "Nimue", loves: ["grape", "carrot"], weight: 540, gender: "f"}
])

learn> db.unicorns.insertMany([
  {name: "Horny", loves: ["carrot", "papaya"], weight: 600, gender: "m", vampires: 63},
  {name: "Aurora", loves: ["carrot", "grape"], weight: 450, gender: "f", vampires: 43},
  {name: "Unicrom", loves: ["enegron", "redbull"], weight: 984, gender: "m", vampires: 182},
  {name: "Roooooodles", loves: ["apple"], weight: 575, gender: "m", vampires: 99},
  {name: "Solnara", loves: ["apple", "carrot", "chocolate"], weight: 550, gender: "f", vampires: 80},
  {name: "Ayna", loves: ["strawberry", "lemon"], weight: 733, gender: "f", vampires: 40},
  {name: "Kenny", loves: ["grape", "lemon"], weight: 690, gender: "m", vampires: 39},
  {name: "Raleigh", loves: ["apple", "sugar"], weight: 421, gender: "m", vampires: 2},
  {name: "Leia", loves: ["apple", "watermelon"], weight: 601, gender: "f", vampires: 33},
  {name: "Pilot", loves: ["apple", "watermelon"], weight: 650, gender: "m", vampires: 54},
  {name: "Nimue", loves: ["grape", "carrot"], weight: 540, gender: "f"}
])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a43b44de42c65eaf0e438dd'),
    '1': ObjectId('6a43b44de42c65eaf0e438de'),
    '2': ObjectId('6a43b44de42c65eaf0e438df'),
    '3': ObjectId('6a43b44de42c65eaf0e438e0'),
    '4': ObjectId('6a43b44de42c65eaf0e438e1'),
    '5': ObjectId('6a43b44de42c65eaf0e438e2'),
    '6': ObjectId('6a43b44de42c65eaf0e438e3'),
    '7': ObjectId('6a43b44de42c65eaf0e438e4'),
    '8': ObjectId('6a43b44de42c65eaf0e438e5'),
    '9': ObjectId('6a43b44de42c65eaf0e438e6'),
    '10': ObjectId('6a43b44de42c65eaf0e438e7')
  }
}
learn> _
```

Рисунок 3 – Добавление 11 документов в коллекцию unicorns методом insertMany()

Документ Dunx был добавлен вторым способом: сначала сформирована переменная document, затем выполнен insertOne(). Метод countDocuments() подтвердил наличие 12 документов.

```
let document = {
  name: "Dunx", loves: ["grape", "watermelon"],
  weight: 704, gender: "m", vampires: 165
}
db.unicorns.insertOne(document)
db.unicorns.countDocuments({})
db.unicorns.find({}, {_id: 0})
```



```
learn> let document = {
  name: "Dunx",
  loves: ["grape", "watermelon"],
  weight: 704,
  gender: "m",
  vampires: 165
}

learn> db.unicorns.insertOne(document)
{
  acknowledged: true,
  insertedId: ObjectId('6a43b4a5e42c65eaf0e438e8')
}

learn> db.unicorns.countDocuments({})
12

learn> db.unicorns.find({}, { _id: 0 })
[
  {
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
```

Рисунок 4 – Добавление документа Dunx вторым способом и проверка количества документов

```

    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  },
  {
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  }
]
learn>

```

Рисунок 5 – Окончание вывода содержимого коллекции *unicorns* после заполнения

Таким образом, исходная коллекция содержит 12 документов и готова к выполнению запросов.

2.2. Сортировка, ограничение и проекция

Сформированы отдельные списки самцов и самок. Результаты отсортированы по полю *name*, а выборка самок ограничена первыми тремя документами.

```

db.unicorns.find({gender: "m"}, {_id: 0}).sort({name: 1})
db.unicorns.find({gender: "f"}, {_id: 0}).sort({name: 1}).limit(3)

```

```

learn> db.unicorns.find(
|   { gender: "m" },
|   { _id: 0 }
| ).sort({ name: 1 })
[
  {
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  }
]
learn>

```

Рисунок 6 – Вывод списка самцов единорогов с сортировкой по имени

```

learn> db.unicorns.find(
|   { gender: "f" },
|   { _id: 0 }
| ).sort({ name: 1 }).limit(3)
[
  {
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  }
]
learn>

```

Рисунок 7 – Вывод первых трёх самок единорогов с сортировкой по имени

Далее найдены все самки, предпочитающие carrot. Методы findOne() и limit(1) продемонстрировали два способа ограничения результата одной особью.

```

db.unicorns.find({gender: "f", loves: "carrot"}, { id: 0})
db.unicorns.findOne({gender: "f", loves: "carrot"}, { _id: 0})
db.unicorns.find({gender: "f", loves: "carrot"}, { _id: 0}).limit(1)

```

```

learn> db.unicorns.find(
|   { gender: "f", loves: "carrot" },
|   { _id: 0 }
| )
[
  {
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
learn> db.unicorns.findOne(
|   { gender: "f", loves: "carrot" },
|   { _id: 0 }
| )
{
  name: 'Aurora',
  loves: [ 'carrot', 'grape' ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
learn> db.unicorns.find(
|   { gender: "f", loves: "carrot" },
|   { _id: 0 }
| ).limit(1)
[
  {
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
learn>

```

Рисунок 8 – Поиск самок, предпочитающих carrot, и ограничение выборки методами *findOne()* и *limit()*

При выводе самцов исключены поля `_id`, `loves` и `gender`. В результате отображены только имя, вес и количество убитых вампиров.

```

db.unicorns.find(
  {gender: "m"},
  { _id: 0, loves: 0, gender: 0}
).sort({name: 1})

```

```

learn> db.unicorns.find(
|   { gender: "m" },
|   { _id: 0, loves: 0, gender: 0 }
| ).sort({ name: 1 })
[
  { name: 'Dunx', weight: 704, vampires: 165 },
  { name: 'Horny', weight: 600, vampires: 63 },
  { name: 'Kenny', weight: 690, vampires: 39 },
  { name: 'Pilot', weight: 650, vampires: 54 },
  { name: 'Raleigh', weight: 421, vampires: 2 },
  { name: 'Roooooodles', weight: 575, vampires: 99 },
  { name: 'Unicrom', weight: 984, vampires: 182 }
]
learn> _

```

Рисунок 9 – Вывод списка самцов без полей предпочтений, пола и идентификатора

Параметр \$natural со значением -1 использован для вывода документов в обратном порядке их добавления. Первым отображается Dunx, добавленный последним, а последним - Horny.

```
db.unicorns.find({}, {_id: 0}).sort({$natural: -1})
```

```

learn> db.unicorns.find(
|   {},
|   { _id: 0 }
| ).sort({ $natural: -1 })
[
  {
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  },
  {
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
]

```

Рисунок 10 – Начало вывода документов в обратном порядке добавления

```

{
  name: 'Rooooooodles',
  loves: [ 'apple' ],
  weight: 575,
  gender: 'm',
  vampires: 99
},
{
  name: 'Unicrom',
  loves: [ 'energon', 'redbull' ],
  weight: 984,
  gender: 'm',
  vampires: 182
},
{
  name: 'Aurora',
  loves: [ 'carrot', 'grape' ],
  weight: 450,
  gender: 'f',
  vampires: 43
},
{
  name: 'Horny',
  loves: [ 'carrot', 'papaya' ],
  weight: 600,
  gender: 'm',
  vampires: 63
}
]
learn>

```

Рисунок 11 – Окончание вывода документов в обратном порядке добавления

Оператор \$slice: 1 оставил в массиве loves только первое предпочтение каждого единорога. В проекции также исключено поле _id.

```
db.unicorns.find({}, {_id: 0, name: 1, loves: {$slice: 1}})
```



```
learn> db.unicorns.find(
  {},
  {
    _id: 0,
    name: 1,
    loves: { $slice: 1 }
  }
)
[
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Aurora', loves: [ 'carrot' ] },
  { name: 'Unicrom', loves: [ 'energon' ] },
  { name: 'Rooooooodles', loves: [ 'apple' ] },
  { name: 'Solnara', loves: [ 'apple' ] },
  { name: 'Ayna', loves: [ 'strawberry' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Leia', loves: [ 'apple' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Nimue', loves: [ 'grape' ] },
  { name: 'Dunx', loves: [ 'grape' ] }
]
learn>
```

Рисунок 12 – Вывод имён единорогов и первого элемента массива loves с использованием \$slice

2.3. Логические операторы

С помощью операторов \$gte и \$lte найдены самки весом от 500 до 700 кг включительно: Solnara, Leia и Nimue.

```
db.unicorns.find(
  {gender: "f", weight: {$gte: 500, $lte: 700}},
  {_id: 0}
)
```

```
learn> db.unicorns.find(
|   {
|     gender: "f",
|     weight: { $gte: 500, $lte: 700 }
|   },
|   {
|     _id: 0
|   }
| )
| [
|   {
|     name: 'Solnara',
|     loves: [ 'apple', 'carrot', 'chocolate' ],
|     weight: 550,
|     gender: 'f',
|     vampires: 80
|   },
|   {
|     name: 'Leia',
|     loves: [ 'apple', 'watermelon' ],
|     weight: 601,
|     gender: 'f',
|     vampires: 33
|   },
|   {
|     name: 'Nimue',
|     loves: [ 'grape', 'carrot' ],
|     weight: 540,
|     gender: 'f'
|   }
| ]
learn>
```

Рисунок 13 – Выборка самок весом от 500 до 700 кг с использованием \$gte и \$lte

Оператор \$all применён для поиска самца весом не менее 500 кг, одновременно предпочитающего grape и lemon. Условиям соответствует только Kenny.

```
db.unicorns.find(
  {gender: "m", weight: {$gte: 500}, loves: {$all: ["grape", "lemon"]}},
  {_id: 0}
)
```

```
learn> db.unicorns.find(
  {
    gender: "f",
    weight: { $gte: 500, $lte: 700 }
  },
  {
    _id: 0
  }
)
[
  {
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
learn> db.unicorns.find(
  {
    gender: "m",
    weight: { $gte: 500 },
    loves: { $all: ["grape", "lemon"] }
  },
  {
    _id: 0
  }
)
[
  {
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  }
]
```

Рисунок 14 – Поиск самца, одновременно предпочитающего grape и lemon, с использованием \$all

Оператор \$exists: false позволил найти документ, в котором отсутствует поле vampires. Таким документом является Nimue.

```
db.unicorns.find({vampires: {$exists: false}}, {_id: 0})
```

```
learn> db.unicorns.find(
|   {
|     vampires: { $exists: false }
|   },
|   {
|     _id: 0
|   }
| )
|
| [
|   {
|     name: 'Nimue',
|     loves: [ 'grape', 'carrot' ],
|     weight: 540,
|     gender: 'f'
|   }
| ]
learn> _
```

Рисунок 15 – Поиск документа без поля *vampires* с использованием *\$exists*

Для самцов сформирован упорядоченный список имён с первым предпочтением. Запрос сочетает фильтр, проекцию массива через *\$slice* и сортировку.

```
db.unicorns.find(
  {gender: "m"},
  {_id: 0, name: 1, loves: {$slice: 1}}
).sort({name: 1})
```

```
learn> db.unicorns.find(
|   { gender: "m" },
|   {
|     _id: 0,
|     name: 1,
|     loves: { $slice: 1 }
|   }
| ).sort({ name: 1 })
|
| [
|   { name: 'Dunx', loves: [ 'grape' ] },
|   { name: 'Horny', loves: [ 'carrot' ] },
|   { name: 'Kenny', loves: [ 'grape' ] },
|   { name: 'Pilot', loves: [ 'apple' ] },
|   { name: 'Raleigh', loves: [ 'apple' ] },
|   { name: 'Roooooodles', loves: [ 'apple' ] },
|   { name: 'Unicrom', loves: [ 'energon' ] }
| ]
learn>
```

Рисунок 16 – Упорядоченный список имён самцов с указанием первого предпочтения

3. Вложенные объекты, функции и агрегирование

3.1. Коллекция *towns* и вложенный объект *mayor*

В коллекцию *towns* добавлены три города. Информация о мэре хранится во вложенном объекте *mayor*, а дата последней переписи - в поле *last_sensus* типа *Date*.

```
db.towns.insertMany([
  {name: "Punxsutawney", population: 6200, last_sensus: ISODate("2008-01-31"), ...},
  {name: "New York", population: 22200000, last_sensus: ISODate("2009-07-31"), ...},
  {name: "Portland", population: 528000, last_sensus: ISODate("2009-07-20"), ...}
])
db.towns.find({}, {_id: 0})
```

```
learn> db.towns.insertMany([
  {
    name: "Punxsutawney",
    population: 6200,
    last_sensus: ISODate("2008-01-31"),
    famous_for: ["phil the groundhog"],
    mayor: {
      name: "Jim Wehrle"
    }
  },
  {
    name: "New York",
    population: 22200000,
    last_sensus: ISODate("2009-07-31"),
    famous_for: ["statue of liberty", "food"],
    mayor: {
      name: "Michael Bloomberg",
      party: "I"
    }
  },
  {
    name: "Portland",
    population: 528000,
    last_sensus: ISODate("2009-07-20"),
    famous_for: ["beer", "food"],
    mayor: {
      name: "Sam Adams",
      party: "D"
    }
  }
])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a43b682e42c65eaf0e438e9'),
    '1': ObjectId('6a43b682e42c65eaf0e438ea'),
    '2': ObjectId('6a43b682e42c65eaf0e438eb')
  }
}
learn> db.towns.find({}, {_id: 0 })
[
  {
    name: 'Punxsutawney',
    population: 6200,
    last_sensus: ISODate('2008-01-31T00:00:00.000Z'),
    famous_for: [ 'phil the groundhog' ],
    mayor: { name: 'Jim Wehrle' }
  },
  {
    name: 'New York',
    population: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ 'statue of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  },
  {
    name: 'Portland',
    population: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams', party: 'D' }
  }
]
```

Рисунок 17 – Добавление документов с вложенными объектами в коллекцию *towns*

```
learn> db.towns.find({}, { _id: 0 })
[
  {
    name: 'Punxsutawney',
    population: 6200,
    last_sensus: ISODate('2008-01-31T00:00:00.000Z'),
    famous_for: [ 'phil the groundhog' ],
    mayor: { name: 'Jim Wehrle' }
  },
  {
    name: 'New York',
    population: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ 'statue of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  },
  {
    name: 'Portland',
    population: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams', party: 'D' }
  }
]
learn>
```

Рисунок 18 – Содержимое коллекции towns после добавления городов

Точечная нотация использована для обращения к вложенному полю mayor.party. Первый запрос нашёл город с мэром независимой партии I, второй - город, у мэра которого поле party отсутствует.

```
db.towns.find({"mayor.party": "I"}, {_id: 0, name: 1, mayor: 1})
db.towns.find({"mayor.party": {"$exists": false}}, {_id: 0, name: 1, mayor: 1})
```

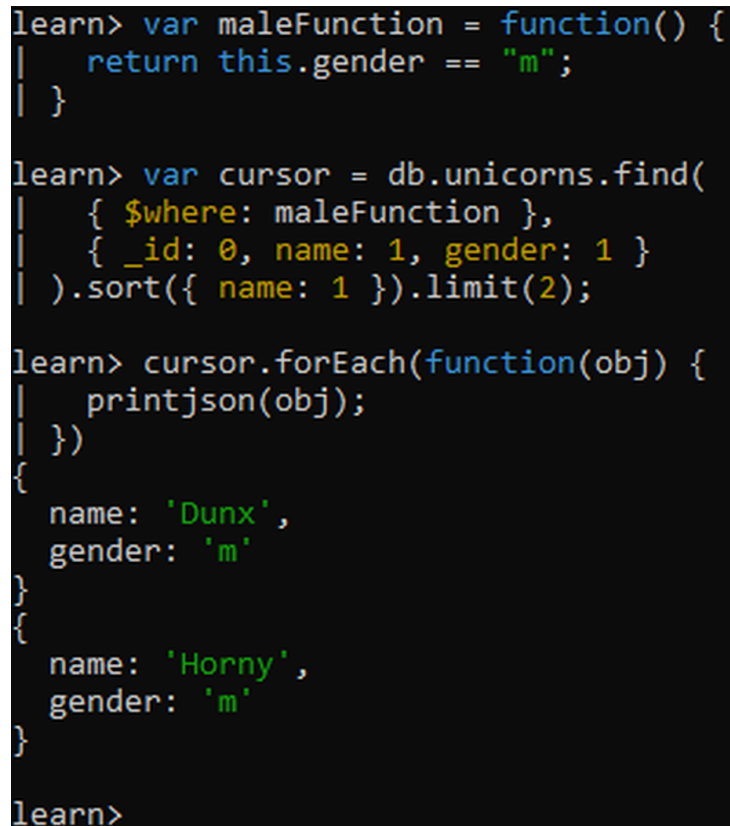
```
learn> db.towns.find(
|   { "mayor.party": "I" },
|   {
|     _id: 0,
|     name: 1,
|     mayor: 1
|   }
| )
|
| [ { name: 'New York', mayor: { name: 'Michael Bloomberg', party: 'I' } } ]
|
learn> db.towns.find(
|   { "mayor.party": { $exists: false } },
|   {
|     _id: 0,
|     name: 1,
|     mayor: 1
|   }
| )
|
| [ { name: 'Punxsutawney', mayor: { name: 'Jim Wehrle' } } ]
learn> _
```

Рисунок 19 – Запросы к вложенному объекту mayor: поиск независимого и беспартийного мэра

3.2. JavaScript-функция и курсор

Сформирована JavaScript-функция `maleFunction` для проверки пола документа. На её основе создан курсор, отсортированный по имени и ограниченный двумя документами. Результат выведен методом `forEach()`.

```
var maleFunction = function() {  
  return this.gender == "m";  
}  
var cursor = db.unicorns.find(  
  {$where: maleFunction}, {_id: 0, name: 1, gender: 1}  
).sort({name: 1}).limit(2);  
cursor.forEach(function(obj) { printjson(obj); })
```



```
learn> var maleFunction = function() {  
|   return this.gender == "m";  
| }  
  
learn> var cursor = db.unicorns.find(  
|   { $where: maleFunction },  
|   { _id: 0, name: 1, gender: 1 }  
| ).sort({ name: 1 }).limit(2);  
  
learn> cursor.forEach(function(obj) {  
|   printjson(obj);  
| })  
{  
  name: 'Dunx',  
  gender: 'm'  
}  
{  
  name: 'Horny',  
  gender: 'm'  
}  
  
learn>
```

Рисунок 20 – Использование JavaScript-функции и курсора для вывода первых двух самцов

3.3. Агрегированные запросы

Метод `countDocuments()` показал, что в коллекции находятся две самки весом от 500 до 600 кг включительно.

```
db.unicorns.countDocuments({  
  gender: "f", weight: {$gte: 500, $lte: 600}  
})
```

```
learn> db.unicorns.countDocuments({
|   gender: "f",
|   weight: { $gte: 500, $lte: 600 }
| })
2
learn>
```

Рисунок 21 – Подсчёт количества самок весом от 500 до 600 кг методом `countDocuments()`

Метод `distinct()` сформировал список всех уникальных элементов массивов `loves`.

```
db.unicorns.distinct("loves")
```

```
learn> db.unicorns.distinct("loves")
[
  'apple',      'carrot',
  'chocolate', 'energon',
  'grape',      'lemon',
  'papaya',     'redbull',
  'strawberry', 'sugar',
  'watermelon'
]
learn> _
```

Рисунок 22 – Вывод уникального списка предпочтений методом `distinct()`

Конвейер `aggregate()` сгруппировал документы по полю `gender` и подсчитал количество документов в каждой группе. Получено 5 самок и 7 самцов.

```
db.unicorns.aggregate([
  {$group: { _id: "$gender", count: {$sum: 1} }},
  {$sort: { _id: 1 } }
])
```



```
learn> db.unicorns.aggregate([
|   {
|     $group: {
|       _id: "$gender",
|       count: { $sum: 1 }
|     }
|   },
|   {
|     $sort: { _id: 1 }
|   }
| ])
| [ { _id: 'f', count: 5 }, { _id: 'm', count: 7 } ]
learn>
```

Рисунок 23 – Группировка единорогов по полу и подсчёт количества документов

4. Изменение данных

4.1. Добавление Barny

В актуальной версии mongosh вместо устаревшего метода `save()` использован `insertOne()`. Добавлен единорог Barny, после чего число документов увеличилось до 13.

```
db.unicorns.insertOne({
  name: "Barny", loves: ["grape"], weight: 340, gender: "m"
})
db.unicorns.find({name: "Barny"}, {_id: 0})
db.unicorns.countDocuments({})
```



```
learn> db.unicorns.aggregate([
|   {
|     $group: {
|       _id: "$gender",
|       count: { $sum: 1 }
|     }
|   },
|   {
|     $sort: { _id: 1 }
|   }
| ])
[ { _id: 'f', count: 5 }, { _id: 'm', count: 7 } ]
learn> db.unicorns.insertOne({
|   name: "Barny",
|   loves: ["grape"],
|   weight: 340,
|   gender: "m"
| })
{
  acknowledged: true,
  insertedId: ObjectId('6a43b79ee42c65eaf0e438ec')
}
learn> db.unicorns.find(
|   { name: "Barny" },
|   { _id: 0 }
| )
[ { name: 'Barny', loves: [ 'grape' ], weight: 340, gender: 'm' } ]
learn> db.unicorns.countDocuments({})
13
learn> _
```

Рисунок 24 – Добавление единорога Barny и проверка количества документов

4.2. Изменение полей и массивов

Оператор `$set` изменил вес Айна на 800 и значение `vampires` на 51. Поля документа, не указанные в операторе, сохранились без изменений.

```
db.unicorns.updateOne(
  {name: "Ayna"},
  {$set: {weight: 800, vampires: 51}}
)
```

```

learn> db.unicorns.updateOne(
|   { name: "Ayna" },
|   {
|     $set: {
|       weight: 800,
|       vampires: 51
|     }
|   }
| )
| {
|   acknowledged: true,
|   insertedId: null,
|   matchedCount: 1,
|   modifiedCount: 1,
|   upsertedCount: 0
| }
learn> db.unicorns.find(
|   { name: "Ayna" },
|   { _id: 0 }
| )
| [
|   {
|     name: 'Ayna',
|     loves: [ 'strawberry', 'lemon' ],
|     weight: 800,
|     gender: 'f',
|     vampires: 51
|   }
| ]
learn>

```

Рисунок 25 – Изменение веса и количества убитых вампиров у Айна оператором \$set

Оператор \$push добавил значение redbull в массив loves единорога Raleigh.

```
db.unicorns.updateOne({name: "Raleigh"}, {$push: {loves: "redbull"}})
```

```

learn> db.unicorns.updateOne(
|   { name: "Raleigh" },
|   {
|     $push: {
|       loves: "redbull"
|     }
|   }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find(
|   { name: "Raleigh" },
|   { _id: 0 }
| )
[
|   {
|     name: 'Raleigh',
|     loves: [ 'apple', 'sugar', 'redbull' ],
|     weight: 421,
|     gender: 'm',
|     vampires: 2
|   }
| ]
learn> _

```

Рисунок 26 – Добавление предпочтения redbull единорогу Raleigh оператором \$push

Метод updateMany() с оператором \$inc увеличил значение vampires на 5 у всех восьми самцов. У Barny поле vampires отсутствовало, поэтому было создано со значением 5.

```

db.unicorns.updateMany(
  {gender: "m"},
  {$inc: {vampires: 5}}
)

```

```

learn> db.unicorns.updateMany(
|   { gender: "m" },
|   {
|     $inc: {
|       vampires: 5
|     }
|   }
| )
| {
|   acknowledged: true,
|   insertedId: null,
|   matchedCount: 8,
|   modifiedCount: 8,
|   upsertedCount: 0
| }
learn> db.unicorns.find(
|   { gender: "m" },
|   {
|     _id: 0,
|     name: 1,
|     vampires: 1
|   }
| ).sort({ name: 1 })
| [
|   { name: 'Barney', vampires: 5 },
|   { name: 'Dunx', vampires: 170 },
|   { name: 'Horny', vampires: 68 },
|   { name: 'Kenny', vampires: 44 },
|   { name: 'Pilot', vampires: 59 },
|   { name: 'Raleigh', vampires: 7 },
|   { name: 'Roooooodles', vampires: 104 },
|   { name: 'Unicrom', vampires: 187 }
| ]
learn>

```

Рисунок 27 – Увеличение значения *vampires* у всех самцов с использованием *\$inc*

Во вложенном объекте *mayor* города *Portland* оператором *\$unset* удалено поле *party*. После изменения мэр считается беспартийным.

```

db.towns.updateOne(
  {name: "Portland"},
  {$unset: {"mayor.party": ""}}
)

```

```

learn> db.towns.updateOne(
  { name: "Portland" },
  {
    $unset: {
      "mayor.party": ""
    }
  }
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.towns.find(
  { name: "Portland" },
  {
    _id: 0,
    name: 1,
    mayor: 1
  }
)
[ { name: 'Portland', mayor: { name: 'Sam Adams' } } ]
learn>

```

Рисунок 28 – Удаление вложенного поля *mayor.party* у города *Portland* оператором *\$unset*

Оператор *\$push* добавил значение *chocolate* в массив предпочтений *Pilot*.

```
db.unicorns.updateOne({name: "Pilot"}, {$push: {loves: "chocolate"}})
```

```

learn> db.unicorns.updateOne(
  { name: "Pilot" },
  {
    $push: {
      loves: "chocolate"
    }
  }
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find(
  { name: "Pilot" },
  { _id: 0 }
)
[
  {
    name: 'Pilot',
    loves: [ 'apple', 'watermelon', 'chocolate' ],
    weight: 650,
    gender: 'm',
    vampires: 59
  }
]
learn>

```

Рисунок 29 – Добавление предпочтения *chocolate* единорогу *Pilot*

Операторы `$addToSet` и `$each` использованы для добавления Aurora сразу двух уникальных предпочтений: `sugar` и `lemon`. В отличие от `$push`, `$addToSet` не создаёт повторяющиеся элементы.

```
db.unicorns.updateOne(
  {name: "Aurora"},
  {$addToSet: {loves: {$each: ["sugar", "lemon"]}}}
)
```

```
learn> db.unicorns.updateOne(
|   { name: "Aurora" },
|   {
|     $addToSet: {
|       loves: {
|         $each: ["sugar", "lemon"]
|       }
|     }
|   }
| )
| {
|   acknowledged: true,
|   insertedId: null,
|   matchedCount: 1,
|   modifiedCount: 1,
|   upsertedCount: 0
| }
learn> db.unicorns.find(
|   { name: "Aurora" },
|   { _id: 0 }
| )
| [
|   {
|     name: 'Aurora',
|     loves: [ 'carrot', 'grape', 'sugar', 'lemon' ],
|     weight: 450,
|     gender: 'f',
|     vampires: 43
|   }
| ]
learn> _
```

Рисунок 30 – Добавление Aurora значений `sugar` и `lemon` с использованием `$addToSet` и `$each`

5. Удаление данных

Из коллекции `towns` удалены документы, в которых у мэра отсутствует поле `party`. Удалено два города: `Punxsutawney` и `Portland`. После проверки оставшегося `New York` коллекция была полностью очищена методом `deleteMany({})`. Метод `countDocuments()` вернул `0`, однако сама пустая коллекция `towns` сохранилась в списке коллекций.

```
db.towns.deleteMany({"mayor.party": {$exists: false}})
db.towns.find({}, {_id: 0})
db.towns.deleteMany({})
db.towns.countDocuments({})
show collections
```

```
learn> db.towns.deleteMany({
|   "mayor.party": { $exists: false }
| })
{ acknowledged: true, deletedCount: 2 }
learn> db.towns.find({}, { _id: 0 })
[
  {
    name: 'New York',
    population: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ 'statue of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  }
]
learn> db.towns.deleteMany({})
{ acknowledged: true, deletedCount: 1 }
learn> db.towns.countDocuments({})
0
learn> show collections
towns
unicorns
learn>
```

Рисунок 31 – Удаление городов с беспартийными мэрами и полная очистка коллекции `towns`

6. Ссылки между документами

Для демонстрации нормализованной модели создана коллекция `habitats` с тремя зонами обитания: `forest`, `mountains` и `coast`. В документах единорогов поле `habitat` содержит DBRef-ссылку на соответствующий документ коллекции `habitats`.

```
db.habitats.insertMany([
  { _id: "forest", full_name: "Enchanted Forest", ... },
  { _id: "mountains", full_name: "Crystal Mountains", ... },
  { _id: "coast", full_name: "Golden Coast", ... }
])
```

```
learn> db.habitats.insertMany([
  {
    _id: "forest",
    full_name: "Enchanted Forest",
    description: "A magical forest with dense vegetation and clean rivers"
  },
  {
    _id: "mountains",
    full_name: "Crystal Mountains",
    description: "High mountains with caves and snowy peaks"
  },
  {
    _id: "coast",
    full_name: "Golden Coast",
    description: "A warm coastal area near the sea"
  }
])
{
  acknowledged: true,
  insertedIds: { '0': 'forest', '1': 'mountains', '2': 'coast' }
}
learn> db.habitats.find()
[
  {
    _id: 'forest',
    full_name: 'Enchanted Forest',
    description: 'A magical forest with dense vegetation and clean rivers'
  },
  {
    _id: 'mountains',
    full_name: 'Crystal Mountains',
    description: 'High mountains with caves and snowy peaks'
  },
  {
    _id: 'coast',
    full_name: 'Golden Coast',
    description: 'A warm coastal area near the sea'
  }
]
learn> db.unicorns.updateOne(
  { name: "Horny" },
  {
    $set: {
      habitat: {
        $ref: "habitats",
        $id: "forest"
      }
    }
  }
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> _
```

Рисунок 32 – Создание коллекции `habitats` и добавление первой DBRef-ссылки

Ссылки назначены единорогам `Horny`, `Aurora`, `Unicrom` и `Pilot`. Запрос по наличию поля `habitat` подтвердил сохранение ссылок.

```
db.unicorns.updateOne({name: "Horny"}, {$set: {habitat: {$ref: "habitats", $id: "forest"}}})
...
db.unicorns.find({habitat: {$exists: true}}, {_id: 0, name: 1, habitat: 1}).sort({name: 1})
```

```

mongosh mongodb://localhost:27017/?directConnection=true&serverSelectionTimeoutMS=200
    }
    }
  }
}
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.updateOne(
  { name: "Unicrom" },
  {
    $set: {
      habitat: {
        $ref: "habitats",
        $id: "mountains"
      }
    }
  }
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.updateOne(
  { name: "Pilot" },
  {
    $set: {
      habitat: {
        $ref: "habitats",
        $id: "coast"
      }
    }
  }
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find(
  { habitat: { $exists: true } },
  {
    _id: 0,
    name: 1,
    habitat: 1
  }
).sort({ name: 1 })
[
  { name: 'Aurora', habitat: DBRef('habitats', 'forest') },
  { name: 'Horny', habitat: DBRef('habitats', 'forest') },
  { name: 'Pilot', habitat: DBRef('habitats', 'coast') },
  { name: 'Unicrom', habitat: DBRef('habitats', 'mountains') }
]
learn>

```

Рисунок 33 – Добавление DBRef-ссылок единорогам и вывод связанных документов

В mongosh 2.9.1 объект DBRef предоставляет имя коллекции через свойство collection, а идентификатор связанного документа - через свойство oid. Ссылка Pilot была разыменована двумя способами; оба запроса вернули документ Golden Coast.

```

var pilot = db.unicorns.findOne({name: "Pilot"})
pilot.habitat.collection
pilot.habitat.oid
db.habitats.findOne({_id: pilot.habitat.oid})
db.getCollection(pilot.habitat.collection).findOne({_id: pilot.habitat.oid})

```

```
learn> pilot.habitat.collection
habitats
learn> pilot.habitat.oid
coast
learn> db.habitats.findOne({
|   _id: pilot.habitat.oid
| })
{
  _id: 'coast',
  full_name: 'Golden Coast',
  description: 'A warm coastal area near the sea'
}
learn> db.getCollection(pilot.habitat.collection).findOne({
|   _id: pilot.habitat.oid
| })
{
  _id: 'coast',
  full_name: 'Golden Coast',
  description: 'A warm coastal area near the sea'
}
learn>
```

Рисунок 34 – Получение документа зоны обитания по DBRef-ссылке единорога Pilot

7. Индексы

Для поля `name` коллекции `unicorns` создан уникальный восходящий индекс `name_1`. Метод `getIndexes()` подтвердил наличие параметра `unique: true`. Попытка повторно вставить документ с именем `Horny` завершилась ожидаемой ошибкой `E11000 duplicate key error`, а количество документов осталось равным 13.

```
db.unicorns.createIndex({name: 1}, {unique: true})
db.unicorns.getIndexes()
db.unicorns.insertOne({name: "Horny", gender: "m"})
db.unicorns.countDocuments({})

learn> db.unicorns.createIndex(
|   { name: 1 },
|   { unique: true }
| )
name_1
learn> db.unicorns.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_', },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
learn> db.unicorns.insertOne({
|   name: "Horny",
|   gender: "m"
| })
MongoServerError: E11000 duplicate key error collection: learn.unicorns index: name_1 dup key: { name: "Horny" }
learn> db.unicorns.countDocuments({})
13
learn> _
```

Рисунок 35 – Создание уникального индекса по полю `name` и проверка запрета дублирования

Выводы

В ходе выполнения лабораторной работы получены практические навыки работы с документной СУБД MongoDB. Создана база данных learn, сформированы коллекции unicorns, towns и habitats. Выполнены операции вставки, выборки, сортировки, ограничения и проекции документов, применены логические операторы и средства работы с массивами.

Освоены запросы к вложенным объектам, использование JavaScript-функций и курсоров, методы countDocuments(), distinct() и конвейер aggregate(). Выполнено изменение одиночных и нескольких документов операторами \$set, \$push, \$inc, \$unset, \$addToSet и \$each, а также удаление выбранных и всех документов коллекции.

Для организации связей между коллекциями использованы DBRef-ссылки, продемонстрировано их разыменовывание. Создан уникальный индекс по полю name и подтверждено, что он предотвращает добавление документов с повторяющимися именами. Все пункты практического задания лабораторной работы 6.2 выполнены в полном объеме.